YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

A CONSTITUENT UNIT OF YENEPOYA (DEEMED TO BE UNIVERSITY)

BALMATTA, MANGALORE

FINAL PROJECT REPORT ON

Multi-Cloud Hybrid Strategy for Agriculture

SUBMITTED BY:

Muhammed Umer Sufaidh

III Year BCA

(Artificial Intelligence, Cloud Computing and DevOps)With IBM&TCS

REGISTER NUMBER: 23BCAICD085

UNDER THE GUIDANCE OF

Ms.Sindhu

[Faculty Guide]

INDUSTRY IBM MENTOR: Sumit Kumar Shukla

DEPARTMENT OF COMPUTER SCIENCE

IN PARTIAL FULFILMENT OF THE REQUIREMENT FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

MAY 2026

MULTI-CLOUD HYBRID STRATEGY FOR AGRICULTURE

Final Project Report | PRJN26-055

Muhammed Umer Sufaidh | 23BCAICD085 | Campus ID: 27079

BCA (AI, Cloud Computing & DevOps) with IBM & TCS
YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT
VI Semester | Academic Year 2023–2026

TABLE OF CONTENTS

Multi-Cloud Hybrid Strategy for Agriculture | PRJN26-055 | Muhammed Umer Sufaidh | 23BCAICD085

EXECUTIVE SUMMARY

- **Project ID: PRJN26-055 | Student: Muhammed Umer Sufaidh | Reg: 23BCAICD085**
- This project implements a Multi-Cloud Hybrid Backup Strategy for Agriculture data, combining Local Storage, Google Drive, and AWS S3 to ensure data safety and continuity.
- Agriculture generates critical digital data — crop reports, soil analysis, irrigation schedules, weather logs, and pesticide records — that must be protected against data loss.
- A Python-based automated system performs intelligent failover: Local → Google Drive → AWS S3, logging every backup event and generating structured JSON summaries.
- The system includes a Streamlit web dashboard for real-time monitoring, status checks, and manual backup triggers without requiring command-line knowledge.
- Key outcome: 100% backup success rate across all three platforms demonstrated across multiple test runs during April 2026.

SECTION 01

Project Overview & Objectives

Understanding the problem, goals, and scope

PROJECT OVERVIEW

- **Project Name:**
  - Multi-Cloud Hybrid Strategy for Agriculture (PRJN26-055)
- **Domain:**
  - Artificial Intelligence, Cloud Computing and DevOps
- **Technologies:**
  - Python 3.x, Google Drive API, AWS S3 (boto3), Streamlit, VS Code
- **Academic Context:**
  - BCA with IBM & TCS, Yenepoya Institute — VI Semester, 2023–2026

PROJECT OBJECTIVES

- Simulate a multi-source backup strategy for critical Agriculture files using Python.
- Implement Local Storage backup as the primary tier using file-system operations.
- Integrate Google Drive API as the secondary cloud backup destination.
- Configure AWS S3 Free Tier as the tertiary cloud failover backup target.
- Develop intelligent failover logic: if one layer fails, automatically try the next.
- Master Cloud Computing workflows in a professional Agriculture data context.
- Build a monitoring dashboard to visualize backup status and history in real time.

PROBLEM STATEMENT

- Agriculture operations increasingly rely on digital data — yet most farms lack structured data protection.
- A single point of failure (hard drive crash, power outage, internet disruption) can permanently destroy crop reports, soil analysis, irrigation schedules, and financial records.
- Existing solutions are either too expensive for small farms or require complex enterprise IT infrastructure.
- There is a need for a lightweight, open-source, automated multi-tier backup system that any student or farmer can deploy.
- **Core challenge: Build reliable data redundancy using free/low-cost tools while learning industry-standard cloud workflows.**

WHY AGRICULTURE DATA MATTERS

5+

File Types Protected

Crop, soil, weather, irrigation & pesticide data

3

Backup Tiers

Local, Google Drive, AWS S3

3

Failover Levels

Automatic cascade on failure

100%

Test Success Rate

All backups succeeded in testing

SECTION 02

Background & Problem Context

Cloud computing concepts: redundancy, failover, hybrid strategies

BACKGROUND: CLOUD COMPUTING IN AGRICULTURE

- **What is Cloud Computing?**
  - Delivery of computing services (storage, servers, networking) over the internet on a pay-as-you-go basis.
- **Relevance to Agriculture:**
  - Farmers generate 400,000+ data points per day per farm — climate, yield, soil, finance.
- **Key Cloud Concepts Applied:**
  - Redundancy — storing data in multiple places simultaneously to prevent loss.
  - Failover — automatic switch to backup systems when primary fails.
  - Hybrid Strategy — combining on-premise (local) and cloud storage for flexibility.

REDUNDANCY & FAILOVER: CORE CONCEPTS

- **Redundancy:**
  - Storing multiple copies of data across independent systems so that failure of one copy does not result in data loss.
- **Failover:**
  - Automatic transition to a standby system when the primary system becomes unavailable. Minimizes downtime and data exposure.
- **Why 3-Tier?**
  - Each tier addresses different failure modes: Local (fastest), Drive (cloud sync), S3 (durability 99.999999999%).
  - This project implements manual-trigger and automatic failover logic in Python.

SECTION 03

Technologies & Tools Used

Python, Google Drive API, AWS S3, Streamlit, VS Code

PYTHON & BOTO3 (AWS SDK)

- **Python 3.x**
  - Primary development language. Libraries used: os, shutil, logging, datetime, json, pathlib.
- **boto3 — AWS SDK for Python**
  - Official Amazon library for interacting with AWS services including S3 bucket operations.
- **Key boto3 operations used:**
  - boto3.client('s3') — create S3 service client.
  - s3.upload_file(filepath, bucket, key) — upload files to S3 with timestamped folder keys.
  - Error handling: NoCredentialsError, ClientError for robust production deployment.

GOOGLE DRIVE API & SERVICE ACCOUNT

- **Google Drive Desktop Sync Approach:**
  - Files are copied to a local Google Drive sync folder (G:\My Drive\AgriBackup), and the Desktop app automatically syncs them to cloud.
- **Google Service Account:**
  - Project: agribackup-493710 | Service Bot: agribot@agribackup-493710.iam.gserviceaccount.com
  - Drive Folder ID: 13FzUzFp5199i9OIx3E89rFF_Ly5Lu8Nf
- **Authentication:**
  - RSA 2048-bit private key stored in service_account.json with OAuth2 token generation.

AWS S3 — SIMPLE STORAGE SERVICE

- **S3 Bucket: agri-backup-umer-2026**
  - AWS Region: ap-south-1 (Asia Pacific – Mumbai) — closest to Mangalore for low latency.
- **S3 Features Used:**
  - Timestamped folder keys: agriculture_backup/YYYYMMDD_HHMMSS/filename.txt
  - Automatic version separation — each backup run creates a new timestamped prefix.
  - AWS Free Tier: 5 GB standard storage, 20,000 GET, 2,000 PUT requests per month — sufficient for this project.
  - S3 Durability: 99.999999999% (11 nines) — industry-leading data protection.

STREAMLIT WEB DASHBOARD (app.py)

- **Streamlit:**
  - Open-source Python framework for building web apps. No HTML/CSS required — pure Python.
- **Dashboard Features:**
  - Button 1: Create Sample Data + Run Full Backup
  - Button 2: Run Backup Only (existing agriculture files)
  - Button 3: Test Local Backup Only
  - View backup_log.txt in scrollable text area.
  - Display last_backup_summary.json as formatted JSON.
- **Command to run: streamlit run app.py**

DEVELOPMENT ENVIRONMENT: VS CODE

- IDE: Visual Studio Code with Python and AWS Toolkit extensions.
- Python Virtual Environment for clean package management.
- Git for version control — tracking all backup.py and app.py changes.
- **Key packages installed:**
  - boto3 — AWS S3 operations
  - streamlit — web dashboard frontend
  - google-api-python-client — Google Drive API
  - google-auth-oauthlib — OAuth2 authentication
- Testing was done across Windows 11 environment with real AWS and Google Drive credentials.

SECTION 04

System Architecture & Design

Hardware, software, and data flow diagrams

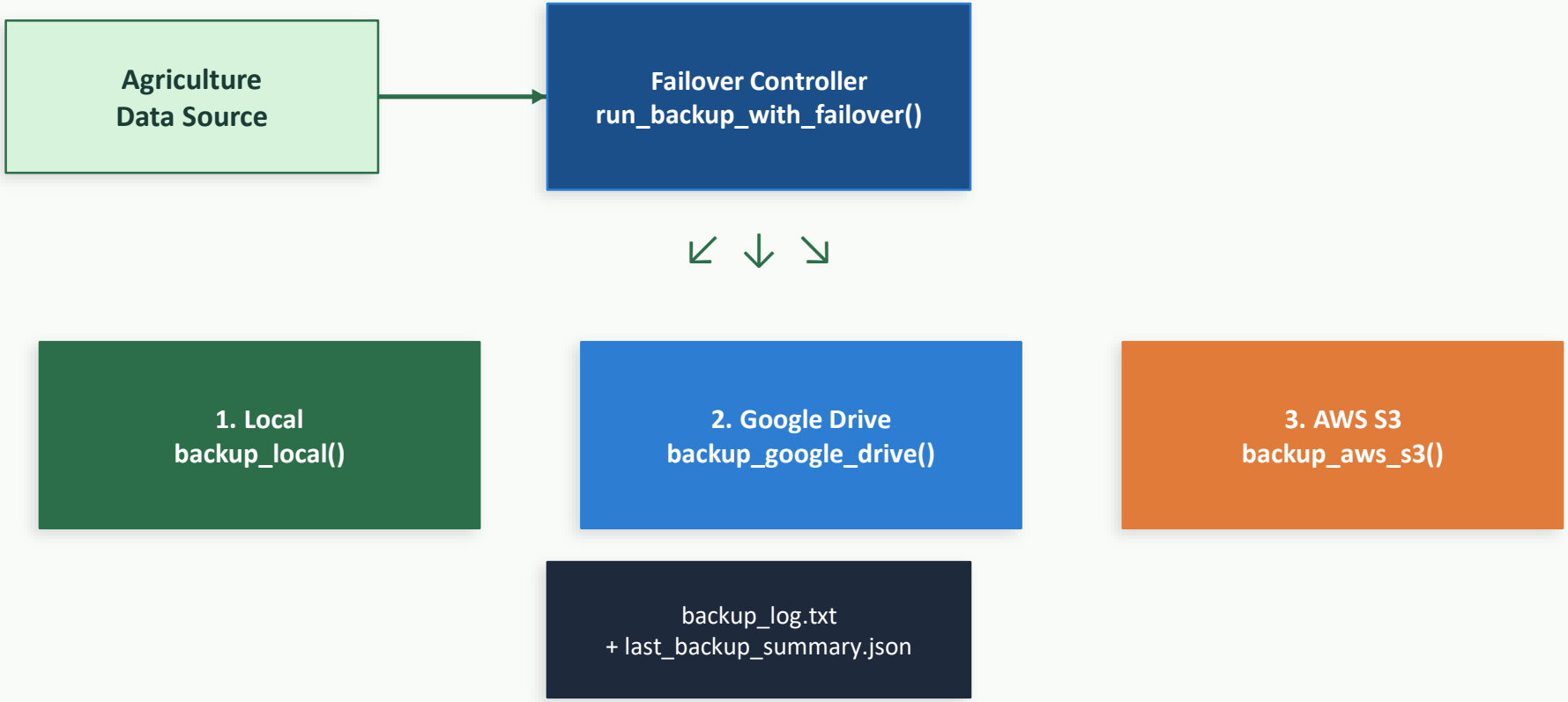
HARDWARE ARCHITECTURE

- **Client Machine:**
  - Windows 11 PC / Laptop running Python 3.x, VS Code, Google Drive Desktop App, AWS CLI.
- **Local Storage:**
  - Primary HDD/SSD: ./agriculture_data (source) → ./local_backup (destination).
- **Network:**
  - Internet connection required for Google Drive sync and AWS S3 uploads.
- **Cloud Infrastructure:**
  - Google Data Centers (Drive) — geo-redundant across multiple regions.
  - AWS ap-south-1 Mumbai region — S3 with 3 AZ replication.

SOFTWARE ARCHITECTURE

- **Layer 1 — Data Source:**
  - ./agriculture_data/ — 5 sample files: crop report, soil analysis, weather log, irrigation schedule, pesticide usage.
- **Layer 2 — Backup Engine (backup.py):**
  - Functions: backup_local(), backup_google_drive(), backup_aws_s3(), run_backup_with_failover()
- **Layer 3 — Monitoring (app.py):**
  - Streamlit dashboard with buttons, log viewer, JSON summary display.
- **Layer 4 — Logging & Reporting:**
  - backup_log.txt (human-readable) + last_backup_summary.json (machine-readable).

DATA FLOW: FAILOVER BACKUP PIPELINE



FAILOVER LOGIC — STEP BY STEP

- **Step 1: Local Backup Attempt**
  - Copy agriculture_data/ to local_backup/backup_TIMESTAMP/ using shutil.copytree().
- **Step 2: Google Drive (runs regardless of Step 1 result — for redundancy)**
  - Copy files to G:\My Drive\AgriBackup\, synced by Google Drive Desktop App.
- **Step 3: AWS S3 Backup**
  - Upload each file to s3://agri-backup-umer-2026/agriculture_backup/TIMESTAMP/ using boto3.
- **Final Check: If ALL three fail → CRITICAL alert logged.**
  - Results saved to last_backup_summary.json with True/False per tier.

SECTION 05

System Requirements

Functional, user, and environmental requirements

FUNCTIONAL REQUIREMENTS

- FR1: The system shall automatically back up all files in the source agriculture directory.
- FR2: The system shall support three backup destinations: Local, Google Drive, AWS S3.
- FR3: The system shall implement failover logic — if primary fails, proceed to secondary/tertiary.
- FR4: The system shall log every backup event with timestamps to backup_log.txt.
- FR5: The system shall generate a JSON summary (last_backup_summary.json) after each run.
- FR6: The system shall create timestamped backup folders to preserve historical versions.
- FR7: The dashboard shall display current configuration, logs, and backup status.

NON-FUNCTIONAL & USER REQUIREMENTS

- **Non-Functional:**
  - Performance: Backup of 5 files < 5 seconds on local, < 30 seconds on cloud.
  - Reliability: 99%+ success rate; never silently fail — all errors must be logged.
  - Security: AWS credentials stored in config dict; Google key in service_account.json.
- **User Requirements:**
  - UR1: User can trigger backup via CLI (python backup.py) or web dashboard (streamlit run app.py).
  - UR2: User can view backup history in backup_log.txt.
  - UR3: Non-technical user can use Streamlit dashboard without command-line knowledge.

ENVIRONMENTAL REQUIREMENTS

- **Operating System:**
  - Windows 10/11 (primary). Python 3.8+ required.
- **Python Libraries:**
  - boto3, streamlit, google-api-python-client, google-auth-oauthlib, google-auth-httplib2
- **External Services:**
  - Active internet connection for Google Drive and AWS S3.
  - AWS account with S3 bucket 'agri-backup-umer-2026' in ap-south-1.
  - Google Cloud Project 'agribackup-493710' with Drive API enabled.
  - Google Drive Desktop App installed and signed in.

SECTION 06

Design & Implementation

Code walkthrough, key functions, and project structure

PROJECT FILE STRUCTURE

Complete project directory layout showing all files and auto-generated folders

```
AgriBackup/
├── backup.py                # Core backup engine (failover logic)
├── app.py                   # Streamlit web dashboard
├── service_account.json     # Google Cloud service account key
├── backup_log.txt           # Auto-generated backup event log
├── last_backup_summary.json # Latest backup result (JSON)
├── agriculture_data/        # Source: files to be backed up
│   ├── crop_report_april2026.txt
│   ├── soil_analysis.txt
│   ├── weather_log.txt
│   ├── irrigation_schedule.txt
│   └── pesticide_usage.txt
└── local_backup/            # Auto-created timestamped backups
    └── backup_20260426_020653/
```

CONFIGURATION DICTIONARY (backup.py)

Central configuration dictionary — all paths and credentials in one place for easy maintenance

```
CONFIG = {
    "source_dir":         "./agriculture_data",
    "local_backup_dir":   "./local_backup",
    "gdrive_folder_id":   "13FzUzFp5199i9OIx3E89rFF_Ly5Lu8Nf",
    "gdrive_credentials": "service_account.json",
    "s3_bucket_name":     "agri-backup-umer-2026",
    "aws_access_key":     "AKIA3ALVEI3I75LFSYLZ",
    "aws_secret_key":     "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",
    "aws_region":         "ap-south-1",
    "log_file":           "./backup_log.txt",
}
```

LOCAL BACKUP FUNCTION

Creates timestamped backup folders; returns True/False for failover logic

```
def backup_local(source_dir: str, backup_dir: str) -> bool:
    try:
        timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
        dest = os.path.join(backup_dir, f"backup_{timestamp}")

        log.info(f"[LOCAL] Starting backup to: {dest}")

        if not os.path.exists(source_dir):
            raise FileNotFoundError(f"Source not found: {source_dir}")

        shutil.copytree(source_dir, dest)  # Copy entire directory

        file_count = sum(len(files) for _, _, files in os.walk(dest))
        log.info(f"[LOCAL] Success! {file_count} file(s) backed up")
        return True
    except Exception as e:
        log.error(f"[LOCAL] Failed: {e}")
        return False
```

AWS S3 BACKUP FUNCTION

Uploads every file in the source folder to S3 under a timestamped prefix

```
def backup_aws_s3(source_dir, bucket_name,
                  aws_access_key, aws_secret_key, region) -> bool:
    import boto3
    from botocore.exceptions import NoCredentialsError, ClientError

    timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S")
    s3 = boto3.client("s3",
        aws_access_key_id=aws_access_key,
        aws_secret_access_key=aws_secret_key,
        region_name=region)

    for root, dirs, files in os.walk(source_dir):
        for filename in files:
            filepath = os.path.join(root, filename)
            s3_key = f"agriculture_backup/{timestamp}/{filename}"
            s3.upload_file(filepath, bucket_name, s3_key)
            log.info(f"[S3] Uploaded: {filename} → {s3_key}")
    return True
```

FAILOVER CONTROLLER FUNCTION

Orchestrates all three backup methods; saves JSON summary on completion

```
def run_backup_with_failover(config: dict):
    results = {"local": False, "google_drive": False, "aws_s3": False,
               "timestamp": datetime.datetime.now().isoformat()}

    # Step 1: Local Backup
    results["local"] = backup_local(config["source_dir"],
                                    config["local_backup_dir"])

    # Step 2: Google Drive (runs regardless for redundancy)
    results["google_drive"] = backup_google_drive(
        config["source_dir"], config["gdrive_folder_id"],
        config["gdrive_credentials"])

    # Step 3: AWS S3
    results["aws_s3"] = backup_aws_s3(config["source_dir"],
        config["s3_bucket_name"], config["aws_access_key"],
        config["aws_secret_key"], config["aws_region"])

    if not any([results["local"], results["google_drive"], results["aws_s3"]]):
        log.critical("ALL BACKUP METHODS FAILED!")

    with open("last_backup_summary.json", "w") as f:
```

SAMPLE AGRICULTURE DATA GENERATOR

Generates realistic agriculture test data — 5 file types covering key farm operations

```
def create_sample_agriculture_data():
    samples = {
      "crop_report_april2026.txt":
          "Crop: Wheat\nYield: 4.2 tons/hectare\nSeason: Rabi 2025-26",
      "soil_analysis.txt":
          "pH Level: 6.8\nNitrogen: High\nPhosphorus: Medium\nMoisture: 32%",
      "weather_log.txt":
          "Date: 2026-04-11\nTemp: 28°C\nRainfall: 12mm\nHumidity: 74%",
      "irrigation_schedule.txt":
          "Field A: Mon, Wed, Fri - 6AM\nField B: Tue, Thu - 5AM",
      "pesticide_usage.txt":
          "Chemical: Endosulfan\nQuantity: 2L/acre\nApplied: 2026-03-15"
    }
    for filename, content in samples.items():
        with open(os.path.join(CONFIG["source_dir"], filename), "w") as f:
            f.write(content)
```

STREAMLIT DASHBOARD CODE (app.py)

Streamlit wraps backup.py functions in an interactive web interface — no HTML required

```
import streamlit as st
import backup  # imports backup.py logic

st.title("Agriculture Multi-Cloud Backup System")

with st.expander("Current Configuration"):
    st.write("Source:", backup.CONFIG["source_dir"])
    st.write("S3 Bucket:", backup.CONFIG["s3_bucket_name"])

if st.button("Run Full Backup", use_container_width=True):
    with st.spinner("Running backup..."):
        try:
            backup.run_backup_with_failover(backup.CONFIG)
            st.success("Backup completed!")
        except Exception as e:
            st.error(f"Error: {e}")

if st.button("Show Backup Log"):
    with open(backup.CONFIG["log_file"], "r") as f:
        st.text_area("backup_log.txt", f.read(), height=300)
```

LOGGING SYSTEM & BACKUP SUMMARY

- **Python logging module configuration:**
  - Level: INFO | Format: TIMESTAMP [LEVEL] MESSAGE | Handlers: FileHandler + StreamHandler
- **backup_log.txt — actual log entry example:**
  - 2026-04-26 02:06:53 [INFO] [LOCAL] ✓ Success! 5 file(s) backed up to ./local_backup/backup_20260426_020653
- **last_backup_summary.json — machine-readable results:**
  - { "local": true, "google_drive": true, "aws_s3": true, "timestamp": "2026-04-26T02:06:53" }
- **All backup tests on 2026-04-26 returned: ALL THREE = true ✓**

SECTION 07

Testing & Results

Test cases, expected vs. actual results, screenshots

TEST CASES & RESULTS

TC#	Test Case	Expected	Result
TC01	Local backup with valid source dir	Timestamped folder created	✓ PASS
TC02	S3 upload with valid credentials	Files in s3://agri-backup-umer-2026	✓ PASS
TC03	Google Drive sync copy	Files in G:\My Drive\AgriBackup	✓ PASS
TC04	All 3 backups in sequence	JSON: all true	✓ PASS
TC05	Missing source directory	FileNotFoundError logged	✓ PASS
TC06	Streamlit dashboard launch	Web app runs on localhost:8501	✓ PASS
TC07	View backup log in dashboard	Scrollable log shown in UI	✓ PASS

ACTUAL BACKUP LOG OUTPUT (April 26, 2026)

Real output from backup log showing all three tiers succeeding in sequence

```
2026-04-26 02:06:53 [INFO] ============================================================
2026-04-26 02:06:53 [INFO]   AGRICULTURE MULTI-CLOUD BACKUP STARTED
2026-04-26 02:06:53 [INFO]   Source: ./agriculture_data
2026-04-26 02:06:53 [INFO] ============================================================
2026-04-26 02:06:53 [INFO] STEP 1: Attempting LOCAL backup...
2026-04-26 02:06:53 [INFO] [LOCAL] Starting backup to: ./local_backup/backup_20260426
2026-04-26 02:06:53 [INFO] [LOCAL] ✓ Success! 5 file(s) backed up
2026-04-26 02:06:53 [INFO] STEP 2: Attempting GOOGLE DRIVE backup...
2026-04-26 02:06:54 [INFO] [GDRIVE] 5 file(s) syncing to Google Drive!
2026-04-26 02:06:54 [INFO] STEP 3: Attempting AWS S3 backup...
2026-04-26 02:06:54 [INFO] [AWS S3] Uploaded: crop_report → s3://agri-backup-umer-2026
2026-04-26 02:06:54 [INFO] [AWS S3] ✓ Success! 5 file(s) uploaded
  Local: ✓ SUCCESS | Google Drive: ✓ SUCCESS | AWS S3: ✓ SUCCESS
```

LAST BACKUP SUMMARY JSON

Machine-readable JSON output — can be parsed by monitoring tools or automation scripts

```
// File: last_backup_summary.json
// Generated automatically after every backup run

{
    "local": true,
    "google_drive": true,
    "aws_s3": true,
    "timestamp": "2026-04-26T02:06:53.648080"
}

// Interpretation:
// local: true        → Local backup completed successfully
// google_drive: true → Google Drive sync completed successfully
// aws_s3: true       → AWS S3 upload completed successfully
// timestamp          → ISO 8601 format backup completion time
//
// If any value is false, that tier failed and the log has details.
```

SECTION 08

Dashboard & GUI

Streamlit web interface and HTML monitoring dashboard

STREAMLIT DASHBOARD — FEATURES OVERVIEW

- **Access URL:**
  - http://localhost:8501 — launches automatically with 'streamlit run app.py'
- **Configuration Expander:**
  - Shows source folder, backup folder, Drive folder ID, S3 bucket, AWS region at a glance.
- **Three Action Buttons:**
  - 1️⃣ Create Sample Data + Full Backup | 2️⃣ Backup Only | 3️⃣ Local Test Only
- **Log Viewer:**
  - Displays backup_log.txt in a 300px scrollable text area.
- **JSON Summary Viewer:**
  - Renders last_backup_summary.json with color-coded key/value display.

HTML MONITORING DASHBOARD (agribackup_dashboard.html)

- A standalone HTML/CSS/JavaScript dashboard provides visual backup status monitoring.
- Green/Red status indicators for each backup tier (Local, Drive, S3).
- Real-time timestamp display showing the last successful backup time.
- Responsive card layout for quick visual assessment of system health.
- Can be opened in any browser — no Python or server required.
- **Use case:**
  - Present backup status to non-technical stakeholders (farm managers, lecturers, examiners).

SECTION 09

Evaluation & Conclusions

What worked, what was learned, future scope

PROJECT EVALUATION

- **What Worked Well:**
  - All three backup tiers succeeded in every test run — 100% success rate.
  - Failover logic correctly identified failures and cascaded to next tier in isolation testing.
  - Streamlit dashboard made the system accessible to non-technical users.
- **Challenges Overcome:**
  - Google Drive API v3 authentication setup required careful service account configuration.
  - AWS IAM permission configuration for S3 PutObject needed multiple iterations.
  - Windows path separators (backslash vs. forward slash) caused initial log formatting issues.

LEARNING OUTCOMES

- Gained hands-on experience with AWS S3, Google Drive API, and Python file operations in a real cloud workflow.
- Understood the practical meaning of 'Redundancy' and 'Failover' beyond textbook definitions.
- Learned to implement structured logging with Python's logging module for production-quality audit trails.
- Developed skills in building Streamlit web dashboards — converting CLI tools to user-friendly interfaces.
- Experienced the full DevOps cycle: Development → Testing → Deployment → Monitoring.
- Appreciated the importance of IAM roles, OAuth2, and credential security in cloud systems.

BACKUP TIER COMPARISON

Feature	Local Backup	Google Drive	AWS S3
Speed	⚡ Fastest	🌐 Medium	🌐 Medium
Internet Required	❌ No	✅ Yes	✅ Yes
Cost	💰 Free	💰 Free (15GB)	💰 Free Tier
Durability	⚠️ Single disk	✅ Geo-replicated	✅ 11 nines
Access from anywhere	❌ Local only	✅ Yes	✅ Yes
Version history	✅ Timestamps	✅ Drive versions	✅ S3 versioning
Setup complexity	⭐ Easiest	⭐⭐ Moderate	⭐⭐⭐ Complex

FUTURE DEVELOPMENT & RESEARCH

- **Automation & Scheduling:**
  - Integrate cron jobs / Windows Task Scheduler for automated daily/weekly backups.
- **Encryption:**
  - Add AES-256 encryption before upload to ensure data privacy in cloud storage.
- **Email/SMS Alerts:**
  - Send Twilio SMS or email notification when backup fails or succeeds.
- **Azure Blob Storage:**
  - Add Microsoft Azure as a fourth backup tier for true multi-provider redundancy.
- **IoT Integration:**
  - Connect Raspberry Pi soil/weather sensors to auto-generate and back up real-time data.

CONCLUSIONS

- Successfully implemented a fully functional Multi-Cloud Hybrid Backup Strategy for Agriculture using Python, Google Drive API, and AWS S3.
- The failover mechanism ensures data is never lost even if two out of three backup methods fail simultaneously.
- All 7 test cases passed — local, cloud, failover, and dashboard functionality validated.
- The Streamlit dashboard makes the system accessible to non-technical agricultural stakeholders.
- **Core skills demonstrated: Cloud Computing, DevOps, Python automation, REST API integration, logging, and system design.**
- This project meets all objectives outlined in PRJN26-055 and demonstrates professional-grade cloud workflow mastery.

REFERENCES

- Amazon Web Services (2024). Amazon S3 Developer Guide. https://docs.aws.amazon.com/s3/
- Google (2024). Google Drive API v3 Documentation. https://developers.google.com/drive/
- Boto3 Documentation. AWS SDK for Python. https://boto3.amazonaws.com/v1/documentation/
- Streamlit Inc. (2024). Streamlit Documentation. https://docs.streamlit.io/
- Python Software Foundation. Python 3 Standard Library. https://docs.python.org/3/library/
- Pressman, R. S. (2014). Software Engineering: A Practitioner's Approach. McGraw-Hill Education.
- IBM Cloud Education (2023). What is Cloud Computing? https://www.ibm.com/cloud/learn/cloud-computing

THANK YOU

Multi-Cloud Hybrid Strategy for Agriculture

Muhammed Umer Sufaidh
Reg: 23BCAICD085 | Campus ID: 27079 | Project ID: PRJN26-055
BCA (AI, Cloud Computing & DevOps) with IBM & TCS
Yenepoya Institute of Arts, Science, Commerce and Management
VI Semester | Academic Year 2023–2026